ПРАКТИКУМ ПО ЯЗЫКУ С И РАБОТЕ С AI

Лабораторная работа № 4

Процедурная декомпозиция прикладной задачи

Курс	1 курс, 1 семестр	**Формат**	выполнение + защита
Объём	6 академических часов	**Максимум**	10 баллов

Суть работы. Каждый вариант — отдельная прикладная программа со своей целью, входами, вычислениями и правилами решения. Нужно самостоятельно разложить назначенную задачу на функции, определить их интерфейсы и доказать корректность выбранной структуры.

1. Результаты обучения

После выполнения работы вы сможете:

- выделять подзадачи и оформлять их как функции с одной ответственностью;
- определять параметры, возвращаемые значения, предусловия и результат функции;
- организовывать передачу данных без изменяемых глобальных переменных;
- связывать блок-схему, контракты и фактическую структуру программы;
- проверять отдельные функции независимо от пользовательского интерфейса;
- критически проверять предложения AI по декомпозиции и тестированию.

2. Что нужно сдать

Артефакт	Содержание
Папка проекта	main.c, README.md, ai_protocol.md, исходник и PNG двух блок-схем
Программа	решение назначенного варианта с самостоятельно спроектированными функциями
Проект декомпозиции	список функций, их ответственность и таблица контрактов в README.md
Блок-схемы	общая структура программы и подробная логика одной нетривиальной функции
Тестирование	контрольный запуск, граничные случаи и независимые проверки функций
AI-протокол	запрос, предложения, собственные решения и доказательства
Защита	объяснение архитектуры и небольшое изменение программы

Рекомендуемая структура папки

```
lab03_variant_N/
|-- main.c
|-- README.md
|-- ai_protocol.md
|-- flowchart.drawio
`-- evidence/
    |-- build.png
    |-- control_run.png
    |-- boundary_run.png
    |-- flowchart_main.png
    `-- flowchart_function.png
```


OlgaSmolyakova

3. Постановка задачи

Получите у преподавателя номер варианта от 1 до 30. Разработайте программу в соответствии с условием выбранного варианта. Выполните процедурную декомпозицию программы, выделив отдельные этапы решения в функции.

> **Главное требование.** Оцениваться будет не только правильность формул, но и качество процедурной декомпозиции: границы функций, их параметры, возвращаемые результаты, отсутствие дублирования и возможность проверять функции отдельно.

4. Что требуется сделать

- ☐ Прочитать только свой вариант и выписать цель программы, входы, промежуточные данные, результаты и ограничения.
- ☐ До написания кода самостоятельно спроектировать не менее пяти функций помимо main. Не использовать названия этапов из условия как механический шаблон: объединение и разделение подзадач нужно обосновать.
- ☐ Для каждой функции заполнить контракт: назначение, параметры, допустимые значения, возвращаемый результат и побочные эффекты.
- ☐ Нарисовать общую блок-схему программы и подробную блок-схему одной нетривиальной функции с ветвлением или несколькими этапами вычисления.
- ☐ Реализовать программу на C17. main организует сценарий выполнения, но не содержит всю предметную логику целиком.
- ☐ Подобрать контрольный набор, граничные тесты и минимум две независимые проверки отдельных функций.
- ☐ Провести AI-аудит своего проекта декомпозиции и зафиксировать проверенные решения.
- ☐ Выполнить выбранный уровень 7–10, подготовить README.md и защитить работу.

5. Общие технические требования

☐ Стандарт C17; точка входа int main(void).

☐ Не менее пяти собственных функций помимо main; минимум три возвращают вычисленный результат.

☐ Одна ответственность функции должна объясняться одним коротким предложением.

☐ Формула или правило реализуется в одном месте и не копируется в main и другие функции.

☐ Изменяемые глобальные переменные запрещены; константы допускаются.

☐ Расчётные функции не должны одновременно запрашивать данные у пользователя.

☐ Перед делением проверяются ограничения, исключающие нулевой знаменатель.

☐ Не использовать массивы, структуры, динамическую память, goto, system("pause") и conio.h.

☐ Программа собирается без предупреждений.

OlgaSmolyakova

Сборка и запуск в Windows

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion -g main.c -o lab03.exe
.\lab03.exe
```

6. Блок-схемы

Используйте diagrams.net (draw.io), LibreOffice Draw, Pencil Project или другой бесплатный редактор. Сдайте исходник схем и PNG-экспорт.

Блок	Назначение	Применение
Овал / скруглённый прямоугольник	начало или конец	границы программы или функции
Параллелограмм	ввод или вывод	входные данные и итоговый отчёт
Ромб	условие	ветви обязательно подписываются «да» и «нет»
Прямоугольник	обработка	вычисление или присваивание
Прямоугольник с двойными боковыми линиями	предопределённый процесс	вызов функции
Стрелка	поток управления	порядок выполнения

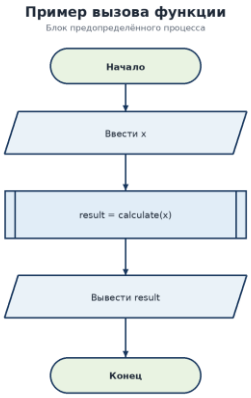


Рисунок 1 — нейтральный пример обозначения вызова функции

Не является решением. Эскиз показывает только способ обозначения вызова функции. Состав блоков, функции и ветвления для своего варианта студент проектирует самостоятельно.

OlgaSmolyakova

7. Индивидуальные варианты

В формулах max(a, b) означает большее из двух значений, min(a, b) — меньшее, |x| — модуль. Фраза «округление вверх» означает наименьшее целое, не меньшее результата деления. Для всех входов задайте и обоснуйте допустимые диапазоны в README.md.

Вариант 1. Диспетчер медицинского дрона

Цель программы	Определить, можно ли выполнить доставку медикаментов дроном.
Входные данные	distance, км; payload, кг; wind, м/с; battery, %; temperature, °C; urgency (1–3).
Что рассчитать	headwind = max(wind, 0); deviation = |temperature − 20|; energy = 2·distance·(1,2 + 0,25·payload + 0,08·headwind + 0,02·deviation); speed = max(12, 36 − 0,7·headwind − 1,1·payload); ETA = distance / speed · 60; risk = 2,5·headwind + 4·payload + 0,4·deviation.
Правила решения	REJECT_SAFETY, если payload > 5 или wind > 14; иначе REJECT_BATTERY, если energy + 15 > battery; иначе REVIEW, если risk > 45 или urgency = 3 и ETA > 20; иначе APPROVED.

Вариант 2. Планировщик поездки электромобиля

Цель программы	Определить готовность электромобиля к поездке и необходимость подзарядки.
Входные данные	distance, км; speed, км/ч; consumption, кВт·ч/100 км; capacity, кВт·ч; charge, %; charger_power, кВт.
Что рассчитать	energy = distance·consumption / 100; available = capacity·charge / 100; travel_time = distance / speed; reserve = available − energy; charge_time = max(0, energy·1,25 − available) / charger_power.
Правила решения	IMPOSSIBLE, если available < energy·1,10; иначе CHARGE_REQUIRED, если available < energy·1,25; иначе LONG_TRIP, если travel_time > 4; иначе READY.

Вариант 3. Управление поливом теплицы

Цель программы	Рассчитать полив и решить, можно ли выполнить его из имеющегося бака.
Входные данные	area, м²; soil, %; target, %; tank, л; flow, л/мин; temperature, °C.
Что рассчитать	moisture_gap = max(0, target − soil); base_water = area·0,6·moisture_gap; evaporation = base_water·max(0, temperature − 25)·0,02; water = base_water + evaporation; time = water / flow; remaining = tank − water.
Правила решения	NO_WATERING, если soil ≥ target; иначе INSUFFICIENT_WATER, если tank < water; иначе LONG_CYCLE, если time > 45; иначе READY.

OlgaSmolyakova

Вариант 4. Планирование партии выпечки

Цель программы	Определить, успеет ли пекарня выполнить заказ имеющимися ресурсами.
Входные данные	orders, шт.; item_weight, г; dough, кг; oven_capacity, шт.; bake_time, мин; deadline, мин.
Что рассчитать	possible_items = целая часть(dough·1000 / item_weight); batches = округление orders / oven_capacity вверх; production_time = batches·bake_time; unused_dough = dough − orders·item_weight / 1000.
Правила решения	NOT_ENOUGH_DOUGH, если possible_items < orders; иначе LATE, если production_time > deadline; иначе WASTE_WARNING, если unused_dough > 2; иначе ACCEPTED.

Вариант 5. Расчёт доставки посылки

Цель программы	Рассчитать стоимость доставки и определить способ обработки заказа.
Входные данные	weight, кг; distance, км; volume, л; urgency (1–3); insured_value, BYN; remote (0/1).
Что рассчитать	base = 5 + 0,8·weight + 0,05·distance + 0,002·volume; urgency_coeff: 1 → 1,0; 2 → 1,35; 3 → 1,75; remote_fee = 12·remote; insurance = 0,01·insured_value; total = base·urgency_coeff + remote_fee + insurance; ETA = distance / 50 + 0,5·remote.
Правила решения	REJECT_OVERSIZE, если weight > 30 или volume > 150; иначе MANUAL_CHECK, если insured_value > 2000; иначе EXPRESS, если urgency = 3; иначе ACCEPTED.

Вариант 6. Оценка тренировочной нагрузки

Цель программы	Оценить интенсивность тренировки и достаточность воды.
Входные данные	age; resting_hr, уд/мин; average_hr, уд/мин; duration, мин; level (1–3); water, мл.
Что рассчитать	max_hr = 220 − age; intensity = (average_hr − resting_hr) / (max_hr − resting_hr)·100; load = intensity·duration / 100; water_need = duration·(6 + 2·level); water_ratio = water / water_need.
Правила решения	STOP, если average_hr ≥ max_hr или intensity > 90; иначе WATER_WARNING, если water_ratio < 0,6; иначе HIGH_LOAD, если load > 50; иначе NORMAL.

OlgaSmolyakova

Вариант 7. Расчёт академической стипендии

Цель программы	Определить категорию стипендии по учебным и социальным показателям.
Входные данные	average_grade (0–10); missed_hours; research_points; volunteer_hours; family_income, BYN; base_amount, BYN.
Что рассчитать	score = average_grade·10 − missed_hours·0,5 + research_points·2 + volunteer_hours·0,2; social_coeff = 1,25 при family_income < 900, иначе 1,0; amount = base_amount·social_coeff; для ENHANCED дополнительно amount = amount·1,4.
Правила решения	DENIED, если average_grade < 6 или missed_hours > 40; иначе ENHANCED, если score ≥ 95; иначе STANDARD, если score ≥ 75; иначе RESERVE. Для RESERVE сумма равна 0.

Вариант 8. Продажа билета в кино

Цель программы	Рассчитать стоимость заказа и проверить возрастные ограничения сеанса.
Входные данные	base_price, BYN; age; age_limit; day (1–7); start_hour; format (2 или 3); snacks, BYN.
Что рассчитать	format_coeff = 1,0 для 2D и 1,35 для 3D; discount = 30% при age < 12, иначе 15% для сеанса до 15:00 в дни 1–5, иначе 0%; ticket = base_price·format_coeff·(1 − discount); total = ticket + snacks.
Правила решения	DENIED_AGE, если age < age_limit; иначе ADULT_REQUIRED, если age < 16 и start_hour ≥ 22; иначе SOLD_3D, если format = 3; иначе SOLD.

Вариант 9. Расчёт счёта за электроэнергию

Цель программы	Рассчитать платёж с учётом тарифа, долга, льготы и просрочки.
Входные данные	day_kwh; night_kwh; debt, BYN; late_days; subsidy, %; warning_limit, BYN.
Что рассчитать	energy_cost = day_kwh·0,25 + night_kwh·0,15; discounted = energy_cost·(1 − subsidy/100); penalty = debt·0,002·late_days; total = discounted + debt + penalty; night_share = night_kwh / (day_kwh + night_kwh)·100.
Правила решения	DISCONNECT_WARNING, если total > warning_limit и late_days > 30; иначе DEBT_WARNING, если debt > 0; иначе NIGHT_EFFICIENT, если night_share ≥ 50; иначе PAYABLE.

OlgaSmolyakova

Вариант 10. Бронирование гостиницы

Цель программы	Определить возможность размещения группы и полную стоимость проживания.
Входные данные	nights; guests; room_capacity; available_rooms; room_rate, BYN; season (1–3); breakfast, BYN/чел.
Что рассчитать	rooms = округление guests / room_capacity вверх; season_coeff: 1 → 0,85; 2 → 1,0; 3 → 1,4; lodging = rooms·nights·room_rate·season_coeff; meals = guests·nights·breakfast; total = lodging + meals; deposit = total·0,30.
Правила решения	NO_ROOMS, если rooms > available_rooms; иначе GROUP_REVIEW, если guests ≥ 10; иначе HIGH_SEASON, если season = 3; иначе CONFIRMED.

Вариант 11. Доходность банковского вклада

Цель программы	Сравнить номинальную и реальную доходность вклада.
Входные данные	principal, BYN; annual_rate, %; months; capitalization (0/1); tax_rate, %; inflation, % годовых.
Что рассчитать	Без капитализации income = principal·annual_rate/100·months/12. С капитализацией: balance = principal; каждый месяц balance = balance·(1 + annual_rate/1200); затем income = balance − principal. tax = max(0, income − 200)·tax_rate/100; final = principal + income − tax; real_final = final / (1 + inflation/100·months/12).
Правила решения	REAL_LOSS, если real_final < principal; иначе TAXABLE, если tax > 0; иначе PROFIT.

Вариант 12. Контроль мобильного тарифа

Цель программы	Рассчитать месячный счёт и оценить соответствие тарифа потреблению.
Входные данные	minutes; messages; data, ГБ; roaming_days; base_fee, BYN; payment, BYN.
Что рассчитать	В пакет входят 300 минут, 100 сообщений и 15 ГБ. over_minutes = max(0, minutes − 300)·0,08; over_messages = max(0, messages − 100)·0,04; over_data = max(0, data − 15)·2,5; roaming = roaming_days·6; total = base_fee + все доплаты; balance = payment − total.
Правила решения	PAYMENT_REQUIRED, если payment < total; иначе CHANGE_PLAN, если total > 1,5·base_fee; иначе ROAMING_NOTICE, если roaming_days > 0; иначе OPTIMAL.

OlgaSmolyakova

Вариант 13. Аренда автомобиля

Цель программы	Проверить возможность аренды и рассчитать её стоимость.
Входные данные	days; distance, км; category (1–3); age; experience, лет; missing_fuel, л; insurance (0/1).
Что рассчитать	daily_rate: 1 → 70; 2 → 110; 3 → 180 BYN; base = days·daily_rate + distance·0,25; young_fee = 0,15·base при age < 25; fuel_fee = missing_fuel·2,2; insurance_fee = 18·days·insurance; total = base + young_fee + fuel_fee + insurance_fee.
Правила решения	DENIED, если age < 21 или experience < 1; иначе HIGH_DEPOSIT, если category = 3 и insurance = 0; иначе FUEL_CHARGE, если missing_fuel > 0; иначе ACCEPTED.

Вариант 14. Запас корма в приюте

Цель программы	Рассчитать потребность в корме и срок, на который хватит запасов.
Входные данные	dogs; average_dog_weight, кг; cats; days; dry_stock, кг; wet_stock, кг.
Что рассчитать	dog_daily = dogs·average_dog_weight·0,02; cat_daily = cats·0,05; daily_need = dog_daily + cat_daily; total_need = daily_need·days; stock = dry_stock + wet_stock; stock_days = stock / daily_need; shortage = max(0, total_need − stock).
Правила решения	CRITICAL, если stock_days < 2; иначе ORDER_REQUIRED, если shortage > 0; иначе LOW_RESERVE, если stock_days < days + 3; иначе ENOUGH.

Вариант 15. Прогноз солнечной электростанции

Цель программы	Спрогнозировать выработку и сравнить её с потреблением объекта.
Входные данные	panels; panel_power, кВт; sun_hours; cloudiness, %; temperature, °C; demand, кВт·ч.
Что рассчитать	cloud_coeff = 1 − cloudiness/100; heat_loss = max(0, temperature − 25)·0,004; production = panels·panel_power·sun_hours·cloud_coeff·(1 − heat_loss); balance = production − demand; coverage = production/demand·100.
Правила решения	DEFICIT, если coverage < 90; иначе RESERVE_LOW, если coverage < 110; иначе SURPLUS, если coverage > 140; иначе BALANCED.

OlgaSmolyakova

Вариант 16. Пополнение складского запаса

Цель программы	Определить момент и объём заказа товара у поставщика.
Входные данные	stock, шт.; daily_demand, шт.; lead_days; safety_days; max_order, шт.; reliability, %.
Что рассчитать	reorder_point = daily_demand·(lead_days + safety_days); expected_during_lead = daily_demand·lead_days; target_stock = reorder_point + daily_demand·safety_days; recommended_order = min(max_order, max(0, target_stock − stock)); days_left = stock / daily_demand.
Правила решения	CRITICAL, если stock < expected_during_lead или reliability < 70; иначе ORDER_NOW, если stock ≤ reorder_point; иначе SUPPLIER_WARNING, если reliability < 85; иначе ENOUGH.

Вариант 17. Расчёт поездки на такси

Цель программы	Рассчитать стоимость поездки, разделить её между пассажирами и проверить оплату.
Входные данные	distance, км; duration, мин; start_hour; passengers; luggage; surge; payment, BYN.
Что рассчитать	base = 3,5 + distance·0,9 + duration·0,12; night_coeff = 1,2 при start_hour < 6 или start_hour ≥ 23, иначе 1; luggage_fee = luggage·1,5; total = base·night_coeff·surge + luggage_fee; per_person = total/passengers; change = payment − total.
Правила решения	DENIED_CAPACITY, если passengers > 4; иначе PAYMENT_REQUIRED, если payment < total; иначе HIGH_SURGE, если surge > 1,5; иначе COMPLETE.

Вариант 18. Подготовка площадки мероприятия

Цель программы	Проверить вместимость, персонал и бюджет мероприятия.
Входные данные	attendees; hall_capacity; duration, ч; available_staff; guests_per_staff; budget, BYN; rent_per_hour, BYN.
Что рассчитать	required_staff = округление attendees / guests_per_staff вверх; rent = duration·rent_per_hour; staff_cost = required_staff·duration·25; total = rent + staff_cost; reserve = budget − total; occupancy = attendees/hall_capacity·100.
Правила решения	NO_CAPACITY, если attendees > hall_capacity; иначе NO_STAFF, если available_staff < required_staff; иначе OVER_BUDGET, если total > budget; иначе HIGH_OCCUPANCY, если occupancy > 90; иначе APPROVED.

OlgaSmolyakova

Вариант 19. Управление отоплением дома

Цель программы	Оценить теплопотери, мощность котла и стоимость обогрева.
Входные данные	area, м²; indoor_temp; outdoor_temp; target_temp; insulation_coeff; boiler_power, кВт; tariff, BYN/кВт·ч.
Что рассчитать	temperature_gap = max(0, target_temp − outdoor_temp); required_power = area·temperature_gap·insulation_coeff / 1000; energy_8h = required_power·8; cost = energy_8h·tariff; power_reserve = boiler_power − required_power.
Правила решения	BOILER_INSUFFICIENT, если power_reserve < 0; иначе OVERHEATED, если indoor_temp > target_temp + 2; иначе HEATING, если indoor_temp < target_temp; иначе COMFORT.

Вариант 20. Анализ поездки на электровелосипеде

Цель программы	Рассчитать стоимость проката, скорость и расход заряда.
Входные данные	minutes; distance, км; bike_type (1/2); battery_start, %; battery_end, %; payment, BYN.
Что рассчитать	unlock_fee: 1 → 1,5; 2 → 3; minute_rate: 1 → 0,12; 2 → 0,20; total = unlock_fee + minutes·minute_rate; average_speed = distance/(minutes/60); battery_used = battery_start − battery_end; consumption = battery_used/distance; change = payment − total.
Правила решения	DATA_ERROR, если battery_end > battery_start; иначе PAYMENT_REQUIRED, если payment < total; иначе SPEED_ALERT, если average_speed > 25; иначе BATTERY_ALERT, если bike_type = 2 и battery_end < 15; иначе COMPLETE.

Вариант 21. Приоритет лабораторного образца

Цель программы	Оценить пригодность образца и определить его место в очереди обработки.
Входные данные	sample_age, ч; storage_temp, °C; urgent (0/1); queue_size; processing_time, мин; reagent_units.
Что рассчитать	quality = 100 − 2·sample_age − 3·|storage_temp − 4|; waiting_time = queue_size·processing_time; finish_time = waiting_time + processing_time; reagent_after = reagent_units − 1.
Правила решения	REJECT_QUALITY, если quality < 60; иначе NO_REAGENT, если reagent_units < 1; иначе PRIORITY, если urgent = 1 или waiting_time > 120; иначе NORMAL_QUEUE.

OlgaSmolyakova

Вариант 22. Оценка доступности кредита

Цель программы	Рассчитать ориентировочный платёж и оценить долговую нагрузку.
Входные данные	income, BYN/мес.; expenses; existing_payments; amount; months; annual_rate, %; down_payment.
Что рассчитать	principal = max(0, amount − down_payment); interest = principal·annual_rate/100·months/12; total_credit = principal + interest; monthly_payment = total_credit/months; debt_load = (existing_payments + monthly_payment)/income·100; free_money = income − expenses − existing_payments − monthly_payment.
Правила решения	DENIED, если debt_load > 50 или free_money < 0; иначе REVIEW, если debt_load > 35; иначе LARGE_DOWN_PAYMENT, если down_payment ≥ 0,4·amount; иначе APPROVED.

Вариант 23. Бронирование столиков ресторана

Цель программы	Проверить возможность размещения гостей и достаточность предоплаты.
Входные данные	guests; table_capacity; available_tables; start_hour; duration, ч; prepayment, BYN; average_check, BYN.
Что рассчитать	tables = округление guests / table_capacity вверх; expected_bill = guests·average_check; required_prepayment = expected_bill·0,20; end_hour = start_hour + duration; prepayment_gap = required_prepayment − prepayment.
Правила решения	NO_SEATS, если tables > available_tables; иначе TOO_LATE, если end_hour > 24; иначе PREPAYMENT_REQUIRED, если prepayment < required_prepayment; иначе GROUP_BOOKING, если guests ≥ 12; иначе CONFIRMED.

Вариант 24. Контроль нагрузки серверного зала

Цель программы	Оценить доступную производительность и энергопотребление серверов.
Входные данные	servers; capacity_per_server, запросов/с; demand; reserve, %; failed_servers; power_per_server, кВт; power_limit, кВт.
Что рассчитать	active = servers − failed_servers; raw_capacity = active·capacity_per_server; usable_capacity = raw_capacity·(1 − reserve/100); load = demand/usable_capacity·100; power = active·power_per_server; spare_capacity = usable_capacity − demand.
Правила решения	CAPACITY_CRITICAL, если demand > usable_capacity; иначе POWER_LIMIT, если power > power_limit; иначе HIGH_LOAD, если load > 80; иначе FAILURE_WARNING, если failed_servers > 0; иначе NORMAL.

OlgaSmolyakova

Вариант 25. Планирование школьного автобусного маршрута

Цель программы	Определить количество автобусов, расход топлива и длительность маршрута.
Входные данные	students; seats_per_bus; available_buses; route_distance, км; stops; fuel_available, л; fuel_rate, л/100 км.
Что рассчитать	required_buses = округление students / seats_per_bus вверх; total_distance = 2·route_distance·required_buses; fuel_need = total_distance·fuel_rate/100; route_time = route_distance/40·60 + stops·2; fuel_left = fuel_available − fuel_need.
Правила решения	NO_BUSES, если required_buses > available_buses; иначе NO_FUEL, если fuel_need > fuel_available; иначе TOO_LONG, если route_time > 90; иначе READY.

Вариант 26. Оценка безопасности туристического маршрута

Цель программы	Оценить время похода, запас светового дня и воды.
Входные данные	distance, км; ascent, м; temperature, °C; daylight, ч; group_speed, км/ч; hikers; water, л.
Что рассчитать	walking_time = distance/group_speed; ascent_time = ascent/300; total_time = walking_time + ascent_time + 0,5; water_need = hikers·total_time·0,4; daylight_reserve = daylight − total_time; water_reserve = water − water_need.
Правила решения	WEATHER_RISK, если temperature < −10 или temperature > 35; иначе NO_DAYLIGHT, если daylight_reserve < 0; иначе WATER_LOW, если water_reserve < 0; иначе HARD_ROUTE, если ascent > 1000; иначе READY.

Вариант 27. Расчёт задолженности библиотеки

Цель программы	Рассчитать штраф читателя и определить состояние его абонемента.
Входные данные	overdue_days; books; daily_rate, BYN; damaged_books; average_book_value, BYN; category (1–3).
Что рассчитать	overdue_fee = overdue_days·books·daily_rate; damage_fee = damaged_books·average_book_value·0,5; discount: category 1 → 0%; 2 → 25%; 3 → 50%; total = (overdue_fee + damage_fee)·(1 − discount); fee_per_book = total/books.
Правила решения	BLOCKED, если overdue_days > 60 или total > 50; иначе DAMAGE_REVIEW, если damaged_books > 0; иначе WARNING, если total > 0; иначе CLEAR.

OlgaSmolyakova

Вариант 28. Прогноз заряда смартфона

Цель программы	Оценить, хватит ли заряда на запланированный режим использования.
Входные данные	capacity, мА·ч; charge, %; screen_hours; active_rate, мА; standby_hours; standby_rate, мА.
Что рассчитать	available = capacity·charge/100; active_need = screen_hours·active_rate; standby_need = standby_hours·standby_rate; total_need = active_need + standby_need; remaining = available − total_need; reserve_percent = remaining/capacity·100; estimated_hours = available / ((active_rate + standby_rate)/2).
Правила решения	NOT_ENOUGH, если remaining < 0; иначе LOW_RESERVE, если reserve_percent < 10; иначе LONG_SESSION, если screen_hours > 6; иначе ENOUGH.

Вариант 29. Проверка заказа интернет-магазина

Цель программы	Рассчитать итоговую стоимость заказа и проверить его перед отправкой.
Входные данные	items_total, BYN; weight, кг; distance, км; promo, %; loyalty_points; payment, BYN; fragile (0/1).
Что рассчитать	actual_promo = min(promo, 30); discount = items_total·actual_promo/100 + min(loyalty_points, items_total·0,20); delivery = 5 + weight·0,6 + distance·0,03 + fragile·8; final = max(0, items_total − discount) + delivery; change = payment − final.
Правила решения	MANUAL_CHECK, если promo > 30 или fragile = 1 и weight > 20; иначе PAYMENT_REQUIRED, если payment < final; иначе FREE_DELIVERY, если items_total − discount ≥ 150; иначе ACCEPTED.

Вариант 30. Контроль параметров аквариума

Цель программы	Оценить состояние воды, фильтрацию и плотность заселения.
Входные данные	volume, л; temperature; target_temperature; pH; fish_count; filter_rate, л/ч; oxygen, мг/л.
Что рассчитать	temperature_deviation = |temperature − target_temperature|; turnover = filter_rate/volume; stocking = fish_count/volume·100; oxygen_reserve = oxygen − 5; recommended_filter = volume·3; filter_gap = recommended_filter − filter_rate.
Правила решения	EMERGENCY, если oxygen < 5 или pH < 6,5 или pH > 8,0; иначе FILTER_LOW, если turnover < 3; иначе OVERSTOCKED, если stocking > 2; иначе TEMP_ADJUST, если temperature_deviation > 2; иначе NORMAL.

8. Проверка программы

Группа тестов	Минимальное требование
Контрольный	обычный набор, для которого заранее вычислены все основные результаты
Границы входов	минимум две нижние/верхние границы и два недопустимых значения
Границы решений	значения непосредственно на пороге двух статусов и по обе стороны от него
Конфликт правил	набор, где истинны минимум два правила; проверить заданный приоритет
Отдельные функции	минимум две функции проверить независимо от полного сценария
Независимый расчёт	для двух тестов показать промежуточные вычисления без копирования кода

Тест зависит от варианта. Студент самостоятельно определяет, какие входы и границы наиболее рискованны именно в его предметной модели. Одинаковая таблица тестов для разных вариантов не засчитывается.

OlgaSmolyakova

9. Обязательная работа с AI

Сначала самостоятельно подготовьте проект декомпозиции и контракты. Затем попросите AI проверить именно ваш вариант. Готовый исходный код у AI не запрашивается.

Рекомендуемый промпт

```
Я выполняю вариант N лабораторной по процедурной декомпозиции на C17.
Цель моей программы: ...
Мой проект функций и контракты: ...

Проверь границы ответственности, параметры и возвращаемые значения.
Не предлагай готовый код и не переписывай решение. Для каждого замечания
предложи тест, которым его можно подтвердить или опровергнуть.
```

Предложение AI	Решение студента	Доказательство
существенное замечание к декомпозиции или контракту	принято / изменено / отклонено	расчёт, тест, правило языка или трассировка
не менее двух дополнительных замечаний	собственное объяснение	конкретный проверяемый результат

Обязательное условие. Хотя бы одно предложение AI нужно обоснованно изменить или отклонить. Полное механическое согласие без проверки не засчитывается.

10. Уровневые задания на 7–10 баллов

Правило. Уровни выполняются накопительно. Для оценки 9 нужны базовая часть и уровни 7, 8 и 9. Расширения не компенсируют неработающую программу.

Уровень 7. Обоснование функций

- Для трёх функций выполните независимую проверку результатов.
- Покажите, какое дублирование или смешение ответственностей предотвращает выбранная структура.
- Укажите функции без побочных эффектов и объясните, почему их удобно тестировать.

Уровень 8. Сценарный анализ

Выберите один значимый числовой параметр своего варианта. Рассчитайте три сценария: исходное значение, уменьшенное и увеличенное значение. Шаг изменения обоснуйте. Повторно используйте разработанные функции и объясните, почему изменились результаты и итоговый статус.

Уровень 9. Автоматические проверки свойств

Сформулируйте не менее четырёх свойств, характерных именно для своего варианта, и реализуйте самопроверку PASS/FAIL. Например, для задачи стоимости может проверяться неотрицательность суммы, а для задачи ресурсов — предсказуемое изменение потребности при росте нагрузки.

Уровень 10. Мутационное тестирование

Создайте отдельную тестовую версию одной функции с одним намеренным логическим дефектом. Заранее укажите нарушаемое свойство и покажите автоматическую проверку, которая обнаруживает мутацию. Испорченная функция не используется в основном расчёте.

OlgaSmolyakova

11. Структура README.md

- ☐ ФИО, группа, номер и название варианта.
- ☐ Цель программы, входы, результаты и самостоятельно определённые ограничения.
- ☐ Проект функций и таблица контрактов.
- ☐ Две блок-схемы и пояснение их соответствия коду.
- ☐ Команды сборки и запуска.
- ☐ Таблица тестов с ожидаемыми и фактическими результатами.
- ☐ Независимые вычисления для двух тестов.
- ☐ AI-протокол с решениями и доказательствами.
- ☐ Результаты уровневых заданий и вывод о декомпозиции.

12. Как формируется оценка

Балл	Что должно быть подтверждено
4	Программа компилируется и частично разделена на функции, но модель, схемы или проверка имеют существенные пробелы.
5	Основной сценарий работает; функции осмысленны, но декомпозиция, контракты или тесты неполны.
6	Полностью выполнена базовая часть: собственный вариант, не менее пяти функций, контракты, две схемы, тесты, README, AI-протокол и защита.
7	Выполнена базовая часть и уровень 7.
8	Выполнены базовая часть и уровни 7–8.
9	Выполнены базовая часть и уровни 7–9.
10	Выполнены все уровни; студент объясняет свойства своей модели, мутацию и самостоятельно изменяет архитектуру или правило на защите.

Ограничения оценки. Один и тот же код с заменой только исходных чисел для разных вариантов не принимается. Без двух блок-схем — максимум 5; без фактической декомпозиции — максимум 5; без защиты — максимум 6.

13. Подготовка к защите

☐ Покажите сборку и контрольный запуск своего варианта.

☐ Сформулируйте цель программы и ответственность каждой функции.

☐ Объясните параметры, результаты и границы выбранных функций.

☐ Свяжите обе блок-схемы с кодом.

☐ Покажите независимый тест функции и граничный тест правила.

☐ Обоснуйте решение по предложению AI.

☐ Будьте готовы объединить две функции или разделить одну и объяснить последствия.

OlgaSmolyakova